Common Loop Algorithms

Tong Yi

Table of contents

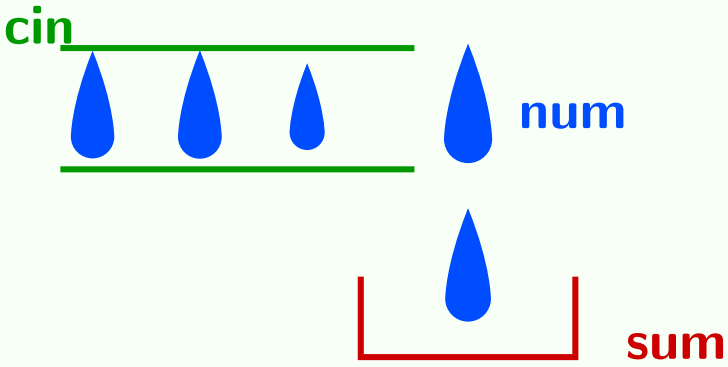
- Sum Up All Integers Entered from Console
- Average Integers Entered from the Console
- Standard Deviation
- Max and Min
- Search Problem
  - Find the First Index of a Character without Using find Method
- Filter and Counting Problems
  - Count Number of Short Words
  - Count Number of Integers Divided by 5
  - Count Number of Spaces
  - Count Occurrences of a Target String within a String

Sum Up All Integers Entered from Console

1. Enter a series of integers from console.
2. End input using ctrl + d (press ctrl key and d at the same time, means end of input) in Mac and ctrl + z in windows.
3. Calculate the sum of all the integers entered.
4. Print the sum.

Sum Up All Integers Entered from Console

1. Enter a series of integers from console.
2. End input using ctrl + d (press ctrl key and d at the same time, means end of input) in Mac and ctrl + z in windows.
3. Calculate the sum of all the integers entered.
4. Print the sum.



Hints: as long as an integer can be extracted from cin, add it to sum.

Code to Sum Up All Integers from Console

code link: https://onlinegdb.com/SP6I4HP4uH

```
std::cout << "Enter integers, then press ctrl + D (on Mac) "
  << "or ctrl + Z (on Windows) to end input:\n";
//Declare variables as close as possible to their first use.
int num;
int sum = 0;
while (std::cin >> num) {
    sum += num;
}
std::cout << "sum: " << sum << std::endl;
```

Condition (`cin >> num`)

- When a **valid** integer is entered, cin successfully stores the number in variable num, and condition ‘(cin ¿¿ num)‘ returns true ☺.

Condition (`cin >> num`)

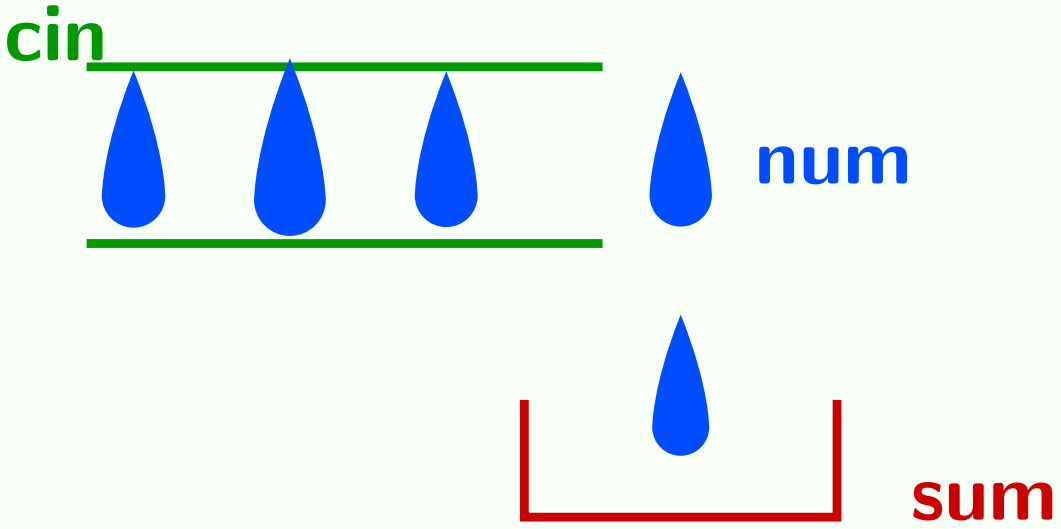
- When a **valid** integer is entered, cin successfully stores the number in variable num, and condition ‘(cin ¿¿ num)‘ returns true ☺.
- When **ctrl + D** (on Unix/Mac) or **ctrl + Z** (on Windows) is entered – signaling the end of input – the stream enters an **end-of-file** state. The extraction ‘(cin ¿¿ num)‘ fails and returns false ☹.

Condition (`cin >> num`)

- When a **valid** integer is entered, cin successfully stores the number in variable num, and condition ‘(cin ¿¿ num)‘ returns true ☺.
- When **ctrl + D** (on Unix/Mac) or **ctrl + Z** (on Windows) is entered – signaling the end of input – the stream enters an **end-of-file** state. The extraction ‘(cin ¿¿ num)‘ fails and returns false ☹.
- When a **non-numeric** character (like "a") is entered where an integer is expected, the **extraction fails** because it cannot be converted. The cin stream enters a failure state and ‘(cin ¿¿ num)‘ returns false ☹.

Average Integers Entered from the Console

- Enter a series of integers from console.
- End input using ctrl + d (press ctrl key and d at the same time, means end of input) in Mac and ctrl + z in windows.
- Calculate the sum of all the integers entered.
- Calculate the average of all the integers entered.
- Print sum and average.



counter

count **EVERY** input

Average Integers from Console: Code

Code link https://onlinegdb.com/r2AU8Jf30.

```
int num;
int sum = 0;


cout << "Enter ints: " << endl;
while (cin >> num) {
    //curly braces are must, since loop
       body has 2 or more stmts
    sum += num;

}
```

Average Integers from Console: Code

Code link https://onlinegdb.com/r2AU8Jf30.

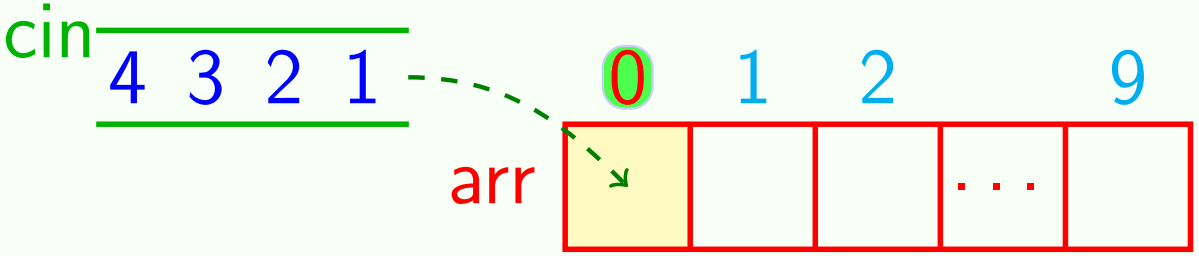
```
int num;
int sum = 0;
int count = 0;

cout << "Enter ints: " << endl;
while (cin >> num) {
    //curly braces are must, since loop
       body has 2 or more stmts
    sum += num;
    count++;
}
```

Standard Deviation

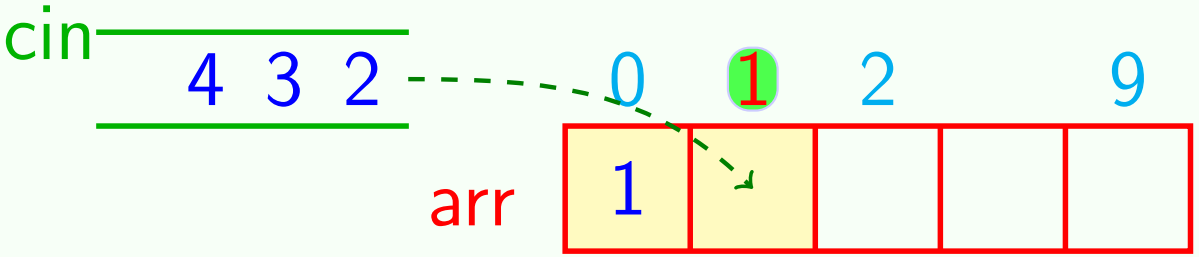
- Enter at most 10 integers from the console, calculate the standard deviation using $\sqrt{(x-\mu)^2/n}$, where x is an element, μ is the average, and n is the number of elements.
  - data: 1, 2, 3, 4
  - number of elements n: 4, mean: $\frac{1+2+3+4}{4} = 2.5$
  - standard deviation: $\sqrt{\frac{(1-2.5)^2+(2-2.5)^2+(3-2.5)^2+(4-2.5)^2}{4}} = 1.11803$
- Calculating standard deviation needs an array.
  - Step 1: find the mean (average)
  - Step 2: compute the squared difference between each data point and that mean.
  - Therefore, all data must be stored in memory, such as an array. This is different from calculating sum in Section 1 or average in Section 2.

Standard Deviation: Save Inputs in Array

- Initially, the array contains no initialized elements. The index of the next cell to be initialized – the first one – is 0.



- Place element 1 in the array's first cell with indexed 0. One element is now initialized. The index of the next cell to write into – the second cell – is 1, which will hold element 2.



Standard Deviation: Code

code link: https://onlinegdb.com/MEsheOmpI

```
const int CAPACITY = 10;
int arr[CAPACITY];
std::cout << "Enter at most " << CAPACITY << " integers: ";
int count = 0; //also serves as index
int num;
int sum = 0;
while ( /* fill in condition*/ ) {
  //TODO: put num in (count)th cell of arr
  //TODO: update sum
  //TODO: update count
}
```

Standard Deviation: Code

code link: https://onlinegdb.com/MEsheOmpI

```
const int CAPACITY = 10;
int arr[CAPACITY];
std::cout << "Enter at most " << CAPACITY
    << " integers: ";
int count = 0; //also serves as index
int num;
int sum = 0;
while (count < CAPACITY && std::cin >>
   num) {
    arr[count] = num;
    sum += num;
    count++;
}
```

Standard Deviation: Code II

code link: https://onlinegdb.com/MEsheOmpI

```
//TODO: calculate the average

//TODO: sum up the square of difference
   of each element and the average


//TODO: calculate stdDev.
//1. Divide the above sum by number of
   elements.
//2. stdDev is the square root of the
   quotient.
```

Standard Deviation: Code II

code link: https://onlinegdb.com/MEsheOmpI

```
double average = 1.0 * sum / count;

double sumDiffSquare = 0;
for (int i = 0; i < count; i++) {
    sumDiffSquare += (arr[i] - average) *
        (arr[i] - average);
}

double stdDev = sqrt(sumDiffSquare /
   count);
```

Find out Max and Min

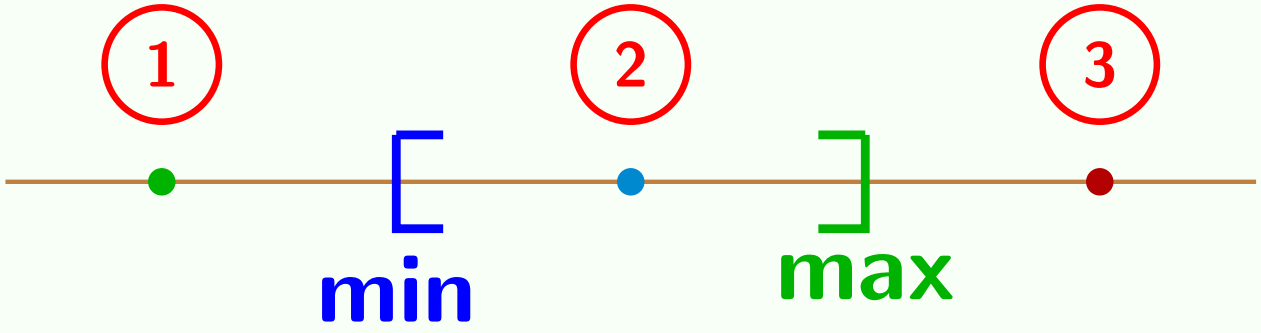
Enter a series of double numbers, find max and min.
Related: babies compete for the cutest model.

- Suppose Alex comes first. Since there is no other competitor, the cutest baby is Alex.
- Now comes Bob. If Bob is cuter than the current cutest baby, change the cutest baby to be Bob.
- Repeat Step 2 for Chris and the rest babies.

Find out Max and Min

Key idea: initialize the max and min value with one of its own values.

1. Initialize max and min to be the first number.
   1.1 What if there is no first number?
2. For each of the remaining inputs, compare it with the current maximum and minimum,



   2.1 what if the current element is larger than the current maximum?
   2.2 otherwise, what if the current element is smaller than the current minimum?
   2.3 what is the last situation?

Find Max and Min: Dode

code link: https://onlinegdb.com/7Ha5jhiUD.

```
double num;
if (/* num cannot be extracted correctly */) {
    std::cerr << "no valid input." << std::endl;
    //TODO: exit to the operating system with error code 1
}

//TODO: declare and set max to be num.
//Warning: what is the type of max?
//TODO: declare and set min to be num.
```

Find Max and Min: Dode

code link: https://onlinegdb.com/7Ha5jhiUD.

```
while (/* num can get a valid input from console */) {
  //TODO: if necessary, update min and max based on num
  if (/* num is larger than ... */) {
    ...
  }
  else if (/* num is smaller than ... */) {
    ...
  }
}
```

Find the First Index of a Character without Using find Method

```
size_t find (char c, size_t pos = 0) const
    noexcept;
```

code link: https://onlinegdb.com/6zgZvY_k7.

```
std::string str("a cat Cat dogcat");
std::size_t found = str.find(' ');

if (found != std::string::npos) {
   std::cout << "space found at: " <<
      found << '\n';
}
else std::cout << "space not found\n";
```

Find the First Index of a Character without Using find Method

Key idea:

1. Declare a bool variable `bFound` indicating whether the target character is found or not. Before search starts, initialize `bFound` to be ...
2. Search elements of the string using index operator [].

```
As long as (...)
begin
    what to do with current character?
end
```

Find the First Index of a Character without Using find Method: code

code link: `https://onlinegdb.com/WUhIlvQAWC`.

Question: can we use `std::cin >> ch;` to input a space to char variable `ch`?

```
std::cout << "Enter a string: ";
std::string str;
getline(std::cin, str);

std::cout << "Enter a character (can be a space) to search: ";
char ch;

```

Find the First Index of a Character without Using find Method: code

code link: `https://onlinegdb.com/WUhIlvQAWC`.

Question: can we use `std::cin >> ch;` to input a space to char variable `ch`?

```
std::cout << "Enter a string: ";
std::string str;
getline(std::cin, str);

std::cout << "Enter a character (can be a space) to search: ";
char ch;
std::cin.get(ch);
```

Find the First Index of a Character without Using find Method: code II

code link: https://onlinegdb.com/WUhIlvQAWC.

```
8  bool bFound = /* fill in a value */;
9  int i = /* fill in a value */;
10 while (/* the target is not found yet and
       there are still elements to be
      searched for */) {
11     What if the current character is the
           target?
12
13     What if the current character is not
           the target?
14 }
```

Find the First Index of a Character without Using find Method: code III

code link: https://onlinegdb.com/WUhIlvQAWC.

```
//Once we are out of the ABOVE while loop
//whose condition is
//( the target is not found yet and there
    are still elements to be searched for
   )
//what can we say?
//either ... or ...

//How do we know the target is found?
//What to do then?
//What to do when the target is not found
   ?
```

Count # of Words with At Most 3 Letters

The container labelled `short_words` is like a sand sifter. It keeps track of small grains of sand.

- What means "a word with at most 3 letters"?
- What should we do for words with three or fewer letters?
- What should we do for all other words?
- What is the expected output?

```
Hi, this is what I mean.
Say hello.^D^D (^D means ctrl + d)
```

Count # of Words with At Most 3 Letters

code link: https://onlinegdb.com/_qU_lWUkNp.

```
int short_words= 0;
std::string word;

//TODO: prompt users to input words
while (std::cin >> word) {
    // >> stops at a white space
    if (word.length() <= 3) {
        short_words++;
    }
}
std::cout << "# of short words: "
  << short_words << std::endl;
```

Count Number of Integers Divided by 5

```
int short_words= 0;
string str;
while (cin >> str)
   { //End input by
    ctrl + D.
  if (str.length()
     <= 3) {
     short_words++;
  }
}
```

```
int divided_by_5 =
   0;
int num;
while (cin >> num)
   {//End input by
   ctrl + D.
  if (num % 5 == 0)
      {
     divided_by_5
        ++;
  }
}
```

Count Number of Spaces in a String

Enter a string, which may contain spaces, from the console. Count and report the number of spaces. code link: `https://onlinegdb.com/EA_LLWPCC`.

```
std::cout << "Enter a string: ";
std::string str;
getline(std::cin, str);
//std::cin >> str;
//stops at first space or return key,
//whichever is encountered first.
//That is, using std::cin >> str;
//str cannot take in a space.
```

Count Number of Spaces in a String: II

May use `isspace` function from <cctype> library.

```
int isspace ( int c );
Check if character is a white-space,
which can a space (' ') or a tab ('\t'),
   a newline ('\n').
```

Count Number of Spaces in a String: II

May use `isspace` function from <cctype> library.

```
int isspace ( int c );
Check if character is a white-space,
which can a space (' ') or a tab ('\t'),
   a newline ('\n').
```

code link: `https://onlinegdb.com/EA_LLWPCC`.

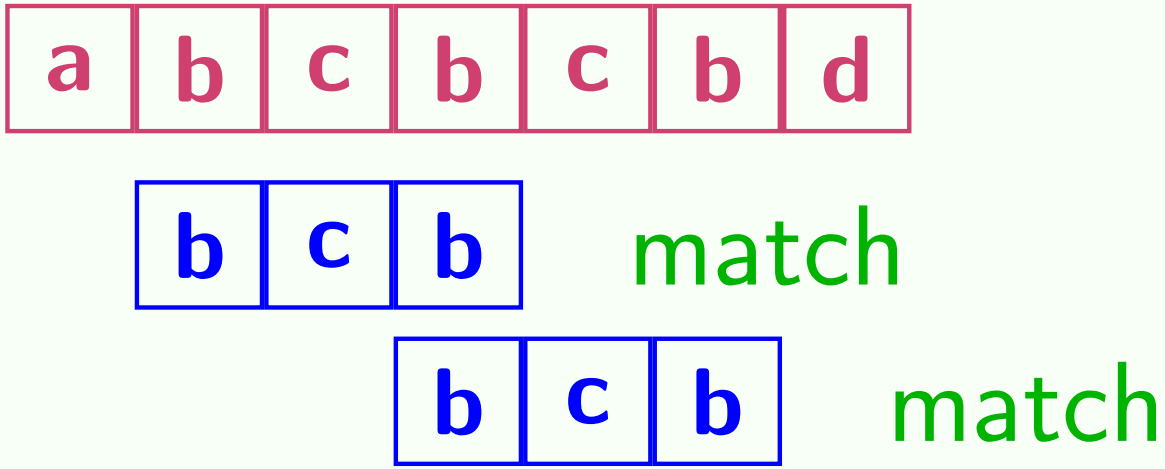
```
int numSpaces = 0;
for (int i = 0; i < str.length(); i++) {
    if (isspace(str[i])) {
        numSpaces++;
    }
}
```

Count Occurrences of a Target String within a String

1. Enter a string, the original string we will search.
2. Enter a target string, the string that we are looking for.
3. Find out how many appearances of the target string in the original string.
4. Find # of appearances of `"bcb"` in `"abcbcbd"`.

Count Occurrences of a Target String within a String

1. Enter a string, the original string we will search.
2. Enter a target string, the string that we are looking for.
3. Find out how many appearances of the target string in the original string.
4. Find # of appearances of "bcb" in "abcbcbd".



Count Occurrences of a Target String within a String: Example

Find # of appearances of "bcb" in "abcbcbd".

0	1	2	3	4	5	6
a	**b**	**c**	**b**	**c**	**b**	**d**

Count Occurrences of a Target String within a String: Example

Find # of appearances of "bcb" in "abcbcbd".

0	1	2	3	4	5	6
a	b	c	b	c	b	d

b	c	b

unmatched

Count Occurrences of a Target String within a String: Example

Find # of appearances of "bcb" in "abcbcbd".

0	1	2	3	4	5	6
a	b	c	b	c	b	d

b	c	b	unmatched
	b	c	b	match

Count Occurrences of a Target String within a String: Example

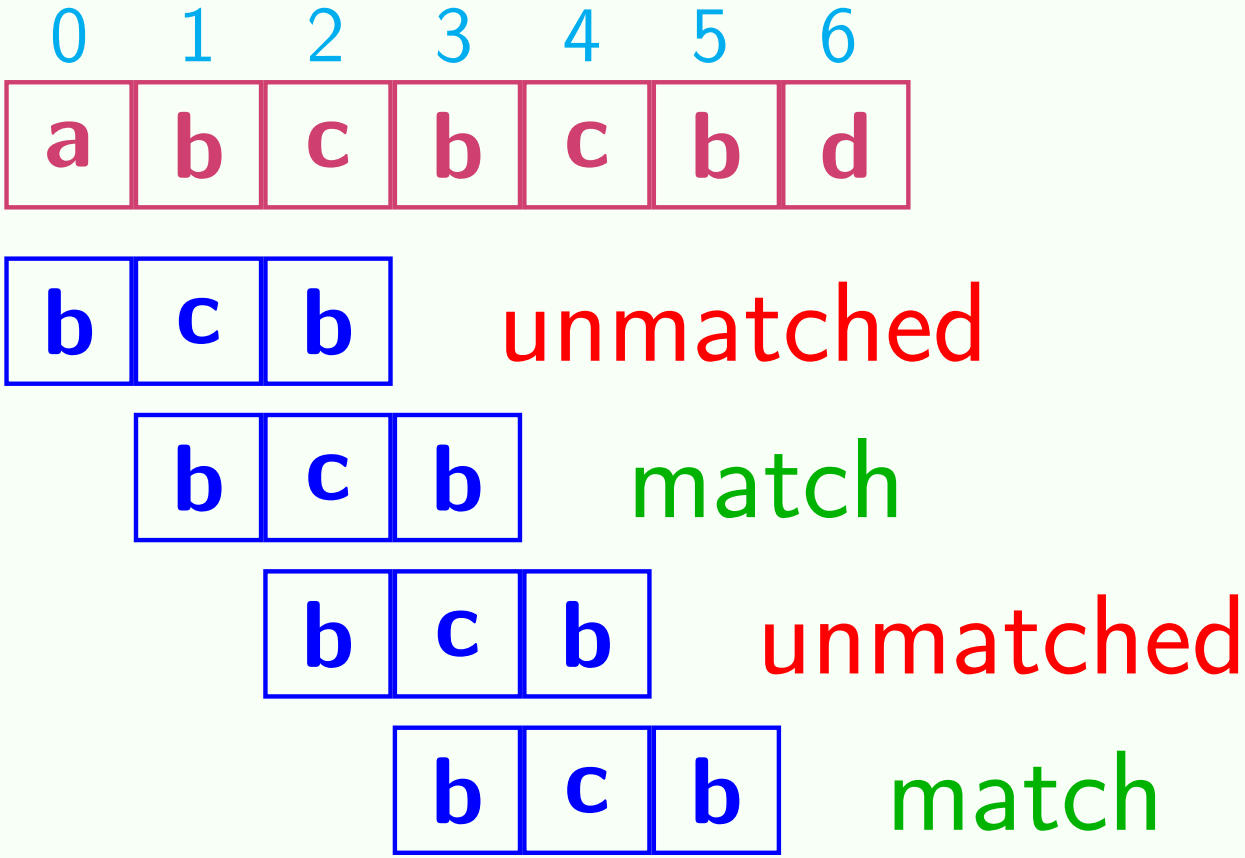
Find # of appearances of "bcb" in "abcbcbd".

0	1	2	3	4	5	6
a	b	c	b	c	b	d

b	c	b					unmatched
	b	c	b				match
		b	c	b			unmatched

Count Occurrences of a Target String within a String: Example

Find # of appearances of "bcb" in "abcbcbd".



Count Occurrences of a Target String within a String: Example

Find # of appearances of "bcb" in "abcbcbd".

0	1	2	3	4	5	6
a	b	c	b	c	b	d

b	c	b					unmatched
	b	c	b				match
		b	c	b			unmatched
			b	c	b		match
				b	c	b	unmatched

Count Occurrences of a Target String within a String: Pseudocode

```
//TODO: enter the string to be searched
//TODO: enter the target string to search

//TODO: track # of occurrences of target
for (/*TODO: for-header*/) {
    //TODO: extract a substring
    //  (1) from which string?
    //  (2) which index to start?
    //  (3) how many character to extract?

    //TODO: what if there is a match?
}
```